

Dekoratory (c.d.)

W kolejnym zestawie zadań (5) zapoznamy się z różnymi technikami optymalizacji wydajnościowej kodu Pythona. Będzie to zarówno bezpośrednia kompilacja do biblioteki wynikowej (.so) za pomocą cython, mypyc, ale również zamiana typów np. heterogenicznej listy Pythona na tablicę określonego typu ndarray z numpy. Będą również prekompilacje za pomocą JIT (just-in-time), które zapisuje się z użyciem dekoratora @jit. Jedną z technik optymalizacji jest też **memoizacja**.

Memoizacja polega na zapamiętywaniu wyników funkcji dla określonych argumentów, aby przy ponownym wywołaniu z tymi samymi argumentami nie wykonywać obliczeń, tylko zwrócić wcześniej zapamiętany wynik.

ZADANIE

Rekurencyjne liczenie ciągu Fibonacciego jest naturalnym algorytmem, niemniej, wyliczanie każdego kolejnego wyrazu ciągu „od początku” jest niepotrzebne. O wiele wydajniejszą metodą byłoby zastosowanie czegoś w rodzaju buforu – pamięci podręcznej, w której zapamiętywalibyśmy poprzednio (wcześniej) wyliczone wyrazy i z nich korzystali. Znacząco przyspieszy to obliczenia. Proszę napisać (uzupełnić poniższy szkielet) kod tak, żeby powstawał słownik – pamięć podręczna z poprzednimi wyliczonymi wartościami i z nich korzystać, a wyliczać nowe tylko gdy jeszcze nie były policzone wcześniej. Słownik proszę zrobić w dekoratorze.

```
import functools
```

```
def pamiec(func):
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        # tu powinien być kod tworzący słownik (element - wartość), który jest sprawdzany
        # do obliczeń wyrazów ciągu - które by były wyliczane rekurencyjnie i wpisywane
        # do słownika tylko gdy wcześniej nie były obliczone
        # normalnie bez buforowania by było return func(*args, **kwargs)

    return wrapper
```

można dodać również pomiar czasu i pokazać jak się on różni, gdy mamy bufor i gdy go nie mamy



```
@pamiec
def fibonaccini(n):
    return n if 0 <= n < 2 else fibonaccini(n - 1) + fibonaccini(n - 2)

for i in range(100):
    print(fibonaccini(i))
```

 ZADANIE – przykładowe rozwiązanie

Rekurencyjne liczenie ciągu Fibonacciego...

```
import functools

⇒ def pamiec(func):
    cache = {} # Słownik do przechowywania wyników obliczeń

    @functools.wraps(func)
    ⇒ def wrapper(*args, **kwargs):
        if args in cache: # Sprawdzenie, czy wynik dla args już istnieje w słowniku
            return cache[args]
        # Oblicz wynik i zapisz go w słowniku
        result = func(*args, **kwargs)
        ⇒ cache[args] = result
        return result

    return wrapper

@pamiec
def fibonacci(n):
    return n if 0 <= n < 2 else fibonacci(n - 1) + fibonacci(n - 2)

for i in range(100):
    print(fibonacci(i))
```